

Dynamic Resource Allocation for Deep Learning Clusters with Separated Compute and Storage

Mingxia Li*, Zhenhua Han[†], Chi Zhang*, Ruiting Zhou[‡], Yuanchi Liu* and Haisheng Tan*

*CAS Key Lab of Wireless-Optical Communications, University of Science and Technology of China, China

[†]Microsoft Research Asia, China

[‡]Southeast University, Nanjing, China

Abstract—The separation of compute and storage in modern cloud services eases the deployment of general applications. However, with the development of accelerators such as GPU/TPU, Deep Learning (DL) training is suffering from potential IO bottlenecks when loading data from storage clusters. Therefore, DL training jobs need to either create local cache in the compute cluster to reduce the bandwidth demands or scale up the IO capacity with higher bandwidth cost. It is full of challenges to choose the best strategy due to the heterogeneous cache/IO preference of DL models, shared dataset among multiple jobs and dynamic GPU scaling of DL training. In this work, we exploit the job characteristics based on their training throughput, dataset size and scalability. For fixed GPU allocation of jobs, we propose CBA to minimize the training cost with a closed-form approach. For clusters that can automatically scale the GPU allocations of jobs, we extend CBA to AutoCBA to support diverse job utility functions and improve social welfare within a limited budget. Extensive experiments with production traces validate that CBA and AutoCBA can reduce IO cost and improve total social welfare by up to 20.5% and 2.27 $\times$, respectively, over the state-of-the-art schedulers for DL training.

I. INTRODUCTION

Modern cloud service providers utilize separated storage and compute. They provide storage services with various types (e.g., blob, file, and page storage) with different redundancy options and IO performance [1], [2]. Also, they provide diverse virtual machines for computing with different purposes, e.g., high-speed disks for IO-intensive application [3], [4] and accelerators for deep learning workloads [5], [6]. When servicing applications, the compute cluster will load/write data from/to the storage cluster. This design allows applications to scale the storage and compute independently, which greatly simplifies the deployment and reduces the cost of general workloads.

However, with the trend of faster GPUs and TPUs and the growing size of the datasets for training Deep Learning (DL) models, the separation of compute and storage resources introduces potential IO bottleneck when loading training data. Fig. 1 demonstrates the DL training process in cloud data-centers. From our observation of a large cloud service provider, over 97% of the training data is placed in storage clusters, which will be loaded on demand when training DL models. Each DL training job loads and consumes data in batches, each of which contains a set of training samples. Due to the pipeline of data loading and computation, once the data loading is slower than the computation on GPU, the training of the DL model will encounter the bottleneck of remote IO from the storage cluster.

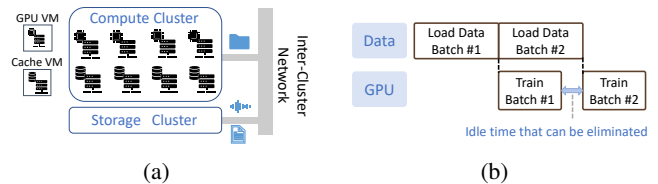


Fig. 1: (a) Separated storage and compute cluster. (b) Training pipeline with IO bottleneck.

There are two solutions to alleviate the remote IO bottleneck from the storage cluster:

- **Creating Cache:** Since DL training repetitively consumes the training dataset multiple times, a DL training job can create CPU VM instances with fast SSD [3] in the same compute cluster to cache the data locally and read it via the fast local network.
- **Increasing Bandwidth:** Some cloud service providers allow users to pay more for higher bandwidth [7]. A DL training job can scale up the bandwidth to the storage cluster beyond its training throughput to avoid IO bottleneck.

Although both strategies alleviate IO bottleneck, they could lead to significant cost differences. In a compute cluster with DL training workloads, jobs with diverse characteristics co-exist. We observed that the cache and bandwidth requirements vary greatly for different types of DL training jobs. For DL models with high training throughput, such as ResNet50 [8] on ImageNet-1k [9] (114MB/s), a small amount of cache (140 GB) is enough to significantly reduce the network bandwidth stress. Conversely, for a slower DL model training on large datasets, such as BERT [10] (2MB/s) on web indexing [11] (1.3TB), which is a big language model that requires a large dataset for training, creating cache is not as efficient as scaling up the bandwidth to the storage cluster. In addition, some jobs sharing the same dataset may reuse the cached data with a higher cache efficiency. Our experiments show that the cost difference between the two strategies could be up to 63.6% of a job's total training cost at the storage level.

However, it is challenging to choose the most efficient strategy. The optimal policy should consider many factors that affect the cache/bandwidth efficiency, including the job training throughput, dataset size, jobs sharing the same dataset, and their scalability to different number of GPUs. Existing resource allocation solutions (e.g., Gandiva [12], Gavel [13])

only allocate compute resources (e.g., GPU) without managing the storage, which could lead to performance degradation due to the ignorance of IO bottleneck.

In this paper, we propose CBA and AutoCBA to jointly allocate compute and storage resources for heterogeneous DL training jobs in GPU clusters. The proposed algorithms can exploit the diverse job characteristics to find the best trade-off among GPUs, cache and bandwidth to optimize the cluster efficiency. Our contributions can be summarized as follows:

- We consider the IO bottleneck problem of DL training in the cloud, which is often ignored by existing GPU allocation algorithms.
- For jobs with a fixed GPU allocation, we derive a novel cache and bandwidth allocation algorithm, named CBA, to minimize the cost. We prove CBA is theoretically optimal. To the best of our knowledge, this is the first algorithm to do the trade-off between bandwidth and cache in the scenario of separated storage and compute service.
- For jobs supporting dynamic scaling of GPU allocation, we extend CBA to AutoCBA to jointly consider IO and computation to pick the optimal combination of cache, bandwidth, GPUs and job-specific utility functions to improve the total social welfare within a limited training budget.
- We implement CBA and AutoCBA and evaluate them using the one-month production trace from Microsoft. The bandwidth and cache allocation significantly reduces the average JCT and GPU cost of the workloads. Compared with the state-of-the-art GPU cluster schedulers, CBA and AutoCBA can reduce IO cost and improve the total social welfare by up to 20.5% and 2.27 $\times$, respectively.

Paper Organization. In Sec. II, we describe our motivations. We formally define the system model and formulate the problem in Sec. III-B. We present our algorithms and analyze their theoretical performance in Sec. IV. The detailed experiment settings and extensive results are shown in Sec. VI. We compare the proposed algorithms with related works in VII. We conclude the paper in Sec. VIII.

II. MOTIVATION

A. IO Bottleneck of Deep Learning Training

The trend in deep learning is towards larger models and a corresponding increase in the size of the datasets used for training. Table. I shows the dataset size growth in one year of tasks from a major cloud service provider. Such a large amount of datasets are generated from production applications, which are stored in dedicated storage cluster in cloud data centers. To meet the increasing demand of DL training, accelerators have been improved greatly over the past ten years. As shown in Table II, compared with the NVIDIA K80 GPU released in 2014, the theoretical FLOPs of the latest NVIDIA GPU A100 has been improved by over 18 $\times$. Using ResNet-50 as the example, the real training speed is improved by over 13 $\times$. However, with the increased FLOPs, the requirements for IO also improve accordingly. Training DL models with data in storage cluster will produce a high volume of traffic to the

Task	Dataset size in 2020	Dataset size in 2021
Task #1	25TB	100TB
Task #2	100GB	1TB
Task #3	100GB	3TB
Task #4	5TB	10TB
Task #5	1.5TB	400TB

TABLE I: Size and growth of the training dataset

GPU	TFLOPS	Training Speed	IO Demand
1*K80	4.1	219 images/s	24 MB/s
1*A100	77.9*	2930 images/s	333 MB/s
8*K80	32.9	1734 images/s	197 MB/s
8*A100	623.2*	16925 images/s	1923 MB/s

* FP16 performance

TABLE II: The training speed and IO demand of ResNet-50 on ImageNet dataset.

network between the storage cluster and the compute cluster. To face the increasing IO demand of DL training, major cloud service providers propose to use cache in the GPU cluster [14] and scale up the network bandwidth of the storage cluster [7].

Model	Dataset Size	BandwidthCost(\$/h)	CacheCost(\$/h)
ResNet50 [8]	645 GB	0.2560	0.1934
AlexNet [15]	140 GB	0.1150	0.0419
YouTube [16]	1.39 TB	0.2753	0.4300
BERT [10]	20 TB	3.9321	6.1439

TABLE III: Cost of eliminating IO bottleneck for DL jobs.

B. Cost Difference of Cache and Bandwidth

Although both options (creating cache or increasing bandwidth) are available to users, we find it is still hard for them to choose the proper cluster setting due to the large and variable cost difference. Based on AWS pricing [3], [7], Table. III demonstrates our cost evaluation of training different jobs either by caching all data locally in the compute cluster or scaling up the IO of the storage cluster beyond the training throughput. Both approaches can alleviate IO bottleneck, but at very different costs. Table III demonstrates the cost of eliminating IO bottleneck for 4 jobs either scaling up the bandwidth of the storage cluster or creating cache VMs in the compute cluster. The cost of training AlexNet with scaled bandwidth is 2 $\times$ of the cache cost. On the contrary, BERT can achieve the ideal training throughput at a much lower cost by scaling bandwidth than using cache VMs.

C. Challenges of Storage Management for DL Training

The root cause of the huge cost difference in storage is the heterogeneous preference for cache and bandwidth of diverse DL training jobs. We observe the bandwidth saved by per GB of cache varies greatly. Fig. 2 demonstrates the cache efficiency (i.e., the bandwidth reduction per GB of cache) of various combinations of DL models and datasets. The heterogeneous cache efficiency of DL jobs mainly results from the DL model size (training speed) and the dataset size. For instance, the cache efficiency of Inception-3 [17]

for ImageNet-1k [9] dataset is $122\times$ higher than the image search model (VLAD) [18] on YouTube dataset [16], because the latter has a larger dataset and lower training speed. In order to allocate storage resources efficiently, the cluster manager should be aware of the heterogeneous cache efficiency. Resource allocation in real clusters is more complicated due to the additional factors to be considered, including:

- **Shared Datasets:** some datasets may be shared by multiple jobs, thus caching these datasets is more efficient;
- **Utility Functions:** jobs in a cluster may have different utility functions (varying reward over job completion time). The allocation of cache and bandwidth needs to consider the trade-off between the cost and the reward received after job completion;
- **Dynamic GPU Scaling:** some clusters can dynamically scale the GPU allocation of a job at runtime, which requires the dynamic allocation of cache and bandwidth accordingly.

However, existing resource allocation solutions (e.g., Gandiva [12], Gavel [13]) only consider the GPU resources without taking cache and bandwidth into account, thus could lead to sub-optimal allocation when IO becomes the bottleneck.

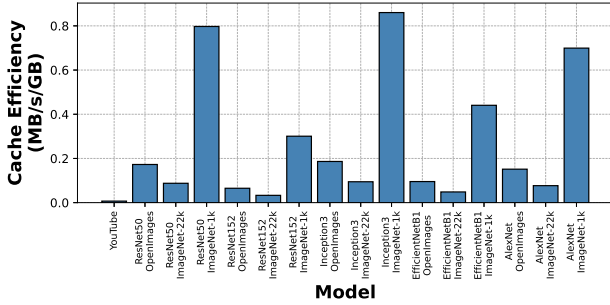


Fig. 2: Bandwidth saved by allocating 1GB cache for jobs.

III. SYSTEM MODEL

A. System Architecture

In the separated storage and compute cluster architecture, training is done on clusters with compute units such as GPUs or TPUs while data is stored on clusters with storage services such as AWS S3. Remote IO bandwidth is required to fetch data from the storage cluster, and local storage resources in the compute cluster can be used as cache. As shown in Fig. 3, the system consists of (1) a profiler to measure the job throughput and (2) an allocator to control the allocation of cache and bandwidth for all jobs. The cache allocator is responsible for creating and destroying cache VMs, whose space is used for storing the training datasets. The bandwidth allocator communicates with the cloud service provider to adjust bandwidth when loading data from the storage cluster. Inside the compute cluster, GPU servers and cache servers co-exist. The DL training jobs running on GPU servers can fetch data from the SSD of the cache servers via the local non-blocking network.

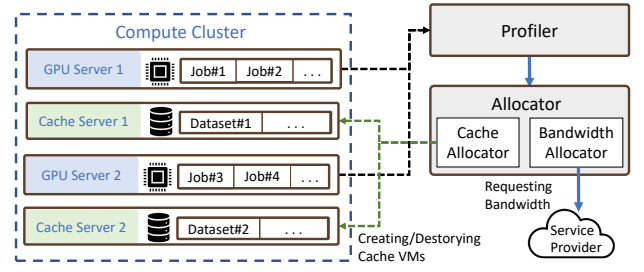


Fig. 3: System architecture.

B. Model

We denote the submitted jobs in the cluster as $J = \{j_1, j_2, \dots, j_n\}$ and d_i as the dataset size for training the job j_i . The dataset ID of j_i is denoted as τ_i . Jobs sharing the same dataset have the same τ_i . A job may train a dataset for multiple epochs. The number of epochs of j_i is denoted as e_i . The GPU number of j_i is denoted as g_i , which can be fixed by users or dynamically scaled during training. Under a specific GPU allocation, the ideal throughput of j_i without IO bottleneck is $f_i^*(g_i)$. A job's ideal throughput can be easily profiled by dry-running it with locally synthesized data to test how fast the GPUs consume the data while training the model.

Users can specify job-specific utility function denoted as $U_i(T_i)$, where T_i is the job completion time of j_i .

For the job j_i , the resource allocator can assign a bandwidth (denoted as b_i) for loading data from the storage cluster. For the dataset d_i , the resource allocator can create cache in the compute cluster, whose cache capacity is denoted as c_i . Jobs using the same dataset can share the cached data items. But the bandwidth allocated to each job is exclusive. Under an allocation of cache, bandwidth and GPU, the training throughput of job j_i is denoted as $f_i(b_i, c_i, g_i)$, which can be calculated as follows according to [19].

Theorem 1. Under a fixed allocation of bandwidth b_i , cache c_i , and GPUs g_i , the real throughput of j_i is

$$f_i(b_i, c_i, g_i) = \min(f_i^*(g_i), \frac{b_i}{1 - c_i/d_i}), \quad (1)$$

Proof. When c_i and b_i is sufficient, j_i is not blocked by IO and reaches its ideal throughput, thus $f_i(b_i, c_i, g_i) = f_i^*(g_i)$. When IO bottleneck happens, on the basis of the unique data access pattern of deep learning jobs: they iterate the whole dataset multiple times (a.k.a. epochs). In each epoch, the data items are shuffled in random order and only accessed once per epoch. Therefore, the probability that an item is accessed is uniform for all data items. Once the cache size is fixed, the probability of a data item storing in the cache is c_i/d_i . Hence the cache miss ratio is $1 - c_i/d_i$. The job requires at least

$$b_i = f_i(b_i, c_i, g_i)(1 - \frac{c_i}{d_i})$$

to achieve the training throughput of $f_i(b_i, c_i, g_i)$. Thus, when there is IO bottleneck, the training throughput of j_i is

$$f_i(b_i, c_i, g_i) = \frac{b_i}{1 - \frac{c_i}{d_i}},$$

which completes the proof. $\square$

The unit prices for cache and bandwidth are denoted as α and β , respectively. The unit GPU price is defined as γ . According to the pricing model of AWS FSx [7], the costs of cache and GPU grow linearly. The cost of bandwidth grows sub-linearly with bandwidth and linearly with dataset size, denoted as $\beta(b_i, d_i)$. Fig. 4 demonstrates the sub-linear bandwidth function of AWS FSx in US East (Ohio). The cost of job j_i is denoted as m_i , which can be calculated as follows.

$$m_i = T_i \cdot (\alpha c_i + \beta(b_i, d_i) + \gamma g_i), \quad (2)$$

where $T_i = \frac{e_i d_i}{f_i(b_i, c_i, g_i)}$ is the job duration of j_i .

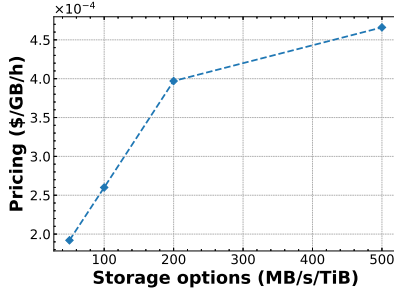


Fig. 4: Sub-linear pricing for bandwidth.

C. Problem Definition

When the number of GPUs is fixed by users, IO bottleneck can occur if the allocated bandwidth or cache is insufficient. For this case, it is equivalent to minimizing the storage cost of eliminating IO bottleneck if the cluster wants to maximize the total social welfare (the difference between utility and cost), which is defined in **Problem 1** as follows.

Problem 1. Given a set of jobs J , and resource prices (α , β and γ), the objective is to find b_i and c_i for each job j_i to minimize the total cost:

$$\min_{b_i, c_i} \sum_i^n m_i. \quad (3)$$

For the case that the GPU number of each job can be adjusted by the allocation algorithm, we define **Problem 2** to maximize total social welfare within a limited budget.

Problem 2. Given a fixed budget M , a set of jobs J , and resource prices (α , β and γ), the objective is to find b_i , c_i and g_i for each job j_i that:

$$\begin{aligned} \max_{b_i, c_i, g_i} \quad & \sum_i^n U_i(e_i \frac{d_i}{f_i(b_i, c_i, g_i)}) - m_i \\ \text{s.t.} \quad & \sum_i^n m_i \leq M, \end{aligned} \quad (4)$$

where the total budget M should be able to cover all jobs executing with at least one GPU.

IV. ALGORITHM

In this section, we first design an algorithm CBA (Cache and Bandwidth Allocation) for allocating cache and bandwidth to eliminate the IO bottleneck when the number of GPUs is pre-specified, which is proven to be optimal for **Problem 1**. Then, we extend CBA to AutoCBA (Storage-aware Job Auto-scaling with CBA) to solve **Problem 2**, i.e., to dynamically scale DL training jobs with the awareness of IO.

A. CBA: Cache and Bandwidth Allocation with Fixed GPUs

The core idea of CBA is to compare two criteria: F_1 and F_2 defined in Algorithm 1, which calculate a job's cost using bandwidth or cache to eliminate the IO bottleneck, respectively. When $F_1 > F_2$, CBA will allocate exclusive bandwidth for the job to eliminate the IO bottleneck. Otherwise, CBA will create cache VMs in the local compute cluster, which is shared by all jobs using the same dataset. The two criteria F_1 and F_2 consider the impact of sharing datasets among multiple jobs, which we prove them to be optimal to solve **Problem 1**. We find the optimal allocation will either only scale up the bandwidth or creating cache VMs to eliminate the IO bottleneck and will never use the mixture of them, which is proven in Theorem 2 and Theorem 3.

Algorithm 1 describes the procedure of CBA. Line 1 iterates the set of running jobs. Line 2 find the set of jobs J_d , which uses the same dataset with j_i . Line 3-4 use all jobs' ideal throughput as bandwidth or the dataset size d_i as cache and get the cost F_1 and F_2 , respectively. F_1 calculates the cost of allocating bandwidth to all jobs in J_d , which is the job duration multiplied by the bandwidth cost $\beta(f_k^*(g_k), d_i)$. F_2 calculates the cost of caching the dataset. The duration of the cache VM is determined by the longest job in J_d .

Line 5-10 compares which type of resource is cheaper, and allocate this type of resource to j_i . When $F_1 > F_2$, the jobs will create cache VMs to store all data locally in the compute cluster. Note that it takes time to make the allocated cache effective, because at the time of cache allocation, there is no data stored in the cache. We discuss the delayed effectiveness of cache in Sec. V-A. We define the effective cache ratio as the fraction of the dataset in the cache that will be used by the running epoch of the training job j_i . For the missed data, bandwidth is still required to fetch them from the storage cluster. Note that the bandwidth is required only when newly allocated cache has not become effective, and in Theorem 2 and Theorem 3, c_i is the already effective cache size.

Next, in Theorem 2 and Theorem 3, we prove the optimality of Algorithm 1 for the cases without and with dataset sharing among multiple jobs, respectively.

Theorem 2. For j_i exclusively using a dataset, the optimal allocation is either $\begin{cases} c_i = 0 \\ b_i = f_i^*(g_i) \end{cases}$ or $\begin{cases} c_i = d_i \\ b_i = 0 \end{cases}$.

Algorithm 1 CBA: Cache and Bandwidth Allocation

Input: J , dataset size d , assigned GPU numbers g , dataset τ , α and β

Output: optimal bandwidth $b = \{b_1, b_2, \dots, b_n\}$ and cache $c = \{c_1, c_2, \dots, c_n\}$

```

1: for  $i = 1$  to  $n$  do
2:    $J_d \leftarrow$  the set of jobs using the same dataset of  $\tau_i$ 
3:    $F_1 \leftarrow \sum_{j_k \in J_d} \frac{e_k d_i}{f_k^*(g_k)} \beta(f_k^*(g_k), d_i)$ 
4:    $F_2 \leftarrow \alpha d_i \cdot \max_{j_k \in J_d} (\frac{e_k d_i}{f_k^*(g_k)})$ 
5:   if  $(F_1 < F_2)$  then
6:      $b_i \leftarrow f_i^*(g_i)$ ,  $c_i \leftarrow 0$ 
7:   else
8:      $ecr_i \leftarrow$  the effective cache ratio of dataset  $\tau_i$ 
9:      $b_i \leftarrow \max(0, (1 - ecr_i) f_i^*(g_i))$ ,  $c_i \leftarrow d_i$ 
10:  end if
11: end for
12: return  $b, c$ 

```

Proof. In (1), given g_i, c_i , the minimum b_i required to obtain the ideal throughput is:

$$b_i = f_i^*(g_i) \left(1 - \frac{c_i}{d_i}\right) \quad (5)$$

Time of an epoch is $\frac{d_i}{f_i(b_i, c_i, g_i)}$. Then, we define the total cost F of a job as:

$$\begin{aligned}
F &= \frac{e_i d_i}{f_i(b_i, c_i, g_i)} (\alpha c_i + \beta(b_i, d_i) + \gamma g_i) \\
&= \frac{e_i d_i}{\min(f_i^*(g_i), \frac{b_i c_i}{1 - \frac{c_i}{d_i}})} (\alpha c_i + \beta(b_i, d_i) + \gamma g_i)
\end{aligned} \quad (6)$$

To minimize F , we insert (5) in (6):

$$F = \frac{e_i d_i}{f_i^*(g_i)} (\alpha c_i + \beta(f_i^*(g_i) (1 - \frac{c_i}{d_i}), d_i) + \gamma g_i) \quad (7)$$

Except c , all other symbols are constants. The derivative of F is:

$$\begin{aligned}
F'(c) &= e_i \left(\frac{\alpha d_i}{f_i^*(g_i)} - \beta'(f_i^*(g_i) (1 - \frac{c_i}{d_i}), d_i) \right) \\
w.r.t. \quad 0 \leq c_i \leq d_i
\end{aligned} \quad (8)$$

We have:

$$\begin{aligned}
F'(0) &= e_i \left(\frac{\alpha d_i}{f_i^*(g_i)} - \beta'(f_i^*(g_i), d_i) \right) \\
F'(d_i) &= e_i \left(\frac{\alpha d_i}{f_i^*(g_i)} - \beta'(0, d_i) \right)
\end{aligned} \quad (9)$$

Since β is sub-linear, we have $\beta'(0, d_i) > \beta'(f_i^*(g_i), d_i) \rightarrow F'(0) > F'(d_i)$. If $F'(d_i) \geq 0 \rightarrow F'(0) \geq 0$, F is non-decreasing along the c dimension. The optimal allocation is $\begin{cases} c_i = 0 \\ b_i = f_i^*(g_i) \end{cases}$. If $F'(0) < 0 \rightarrow F'(d_i) < 0$, F is decreasing.

The optimal allocation is $\begin{cases} c_i = d_i \\ b_i = 0 \end{cases}$. If $F'(0) > 0$ and

$F'(d_i) < 0$, F is increasing then decreasing. The optimal allocation is either $\begin{cases} c_i = 0 \\ b_i = f_i^*(g_i) \end{cases}$ or $\begin{cases} c_i = d_i \\ b_i = 0 \end{cases}$. $\square$

Theorem 3. For a set of jobs J_d sharing the same dataset, the optimal allocation is either $\begin{cases} c_i = 0 \\ b_i = f_i^*(g_i) \end{cases}$ or $\begin{cases} c_i = d_i \\ b_i = 0 \end{cases}$ for $\forall j_i \in J_d$.

Proof. The total cost F_n for n jobs sharing the dataset τ_i :

$$\begin{aligned}
F_n(c) &= \max_{j_i \in J_d} \left(\frac{e_i d_i}{f_i^*(g_i)} \right) \alpha c_i + \\
&\sum_{j_i \in J_d} \frac{e_i d_i}{f_i(b_i, c_i, g_i)} \beta(f_i^*(g_i) (1 - \frac{c_i}{d_i}), d_i) + \sum_{j_i \in J_d} \frac{e_i \gamma g_i d_i}{f_i}
\end{aligned} \quad (10)$$

Similar with the proof of Theorem. 2. The derivative of F_n is:

$$\begin{aligned}
F'_n(c) &= \max_{j_i \in J_d} \left(\frac{e_i d_i}{f_i^*(g_i)} \right) \alpha - \sum_{j_i \in J_d} e_i \beta'(f_i^*(g_i) (1 - \frac{c_i}{d_i}), d_i) \\
w.r.t. \quad 0 \leq c \leq d_i
\end{aligned} \quad (11)$$

We have:

$$\begin{aligned}
F'_n(0) &= \max_{j_i \in J_d} \left(\frac{e_i d_i}{f_i^*(g_i)} \right) \alpha - \sum_{j_i \in J_d} e_i \beta'(f_i^*(g_i), d_i) \\
F'_n(d_i) &= \max_{j_i \in J_d} \left(\frac{e_i d_i}{f_i^*(g_i)} \right) \alpha - |J_d| e_i \beta'(0, d_i)
\end{aligned} \quad (12)$$

Similarly we have $F'_n(0) > F'_n(d_i)$. The proof follows that of Theorem. 2. Note that d_i is only paid for once no matter how many jobs are in J_d . $\square$

B. AutoCBA: Storage-aware Job Auto-scaling

In **Problem 2**, jobs can dynamically scale their GPU allocation at runtime and users can specify the utility function of job completion time. The goal is to maximize the total social welfare (total utility minus cost). When GPU allocation can be dynamically adjusted, the ideal throughput $f_i^*(g_i)$ of j_i varies over the assigned number of GPUs. Therefore, the allocation of cache and bandwidth should be jointly optimized with GPU dynamic scaling in a unified manner.

First, we define the social welfare of the job j_i as follows

$$P_i(b_i, c_i, g_i) = U_i(T_i) - T_i \cdot (\alpha c_i + \beta(b_i, d_i) + \gamma g_i). \quad (13)$$

To extend CBA to **Problem 2**, our main idea of AutoCBA is to greedily add computing resources to the jobs with maximum social welfare and call a simple variant of CBA to eliminate IO bottleneck. AutoCBA aims at maximizing the social welfare only when allocated GPU number is an integer greater than 1, since we iteratively add budget to the job that gains the most social welfare. Algorithm. 2 describes the details of AutoCBA.

Lines 1-4 initialize the allocation for all jobs with at least 1 GPU. CBA algorithm is called to allocate cache and bandwidth for all jobs to eliminate their IO bottleneck within a limited

budget M . Once the cost of cache and bandwidth exceeds the budget M , CBA will not continue to allocate cache and bandwidth to remaining jobs. CBA will update the remaining budget M and the received the social welfare P_i accordingly.

Line 5-16 iteratively use the remaining budget until it is all spent or the social welfare cannot be further improved with more allocation. Line 6-9 tries to add one more GPU to each job to test the benefit of scaling out the training. Line 10-15 finds the job with the most benefit to add one more GPU and corresponding cache/bandwidth to eliminate the IO bottleneck.

Algorithm 2 AutoCBA: Job Auto-scaling with CBA

Input: $J, d, \alpha, \gamma, \beta$, utility function U , budget M , remaining epoch e for each job.

Output: bandwidth b , cache c and GPU g for each job

```

1: Assign each job with 1 GPU.
2:  $M \leftarrow M - \sum_{j_i \in J} \gamma e_i \frac{d_i}{f_i^*(1)}$ .
3: Call CBA w/ a budget  $M$  and update  $M$  and  $P_i (\forall j_i \in J)$ .
4:  $P_k = \max_{j_k \in J} P_i$ 
   // Iteratively add GPU to each job.
5: while  $M > 0$  and  $P_k > 0$  do
6:   for  $j_i \in J$  do
7:     Call CBA with  $M$  for  $g_i + 1$  but NOT update
8:     Record the profit  $P_i(b_i, c_i, g_i + 1)$ 
9:   end for
10:  Find  $j_k$  with the maximum  $P_k$ 
11:  if  $P_k > 0$  then
12:     $g_k \leftarrow g_k + 1$ 
13:     $M \leftarrow M - e_k \frac{d_k}{f_k^*(g_k)} \gamma$ 
14:    Call CBA w/  $M$  and update  $M$  and  $P_i (\forall j_i \in J)$ .
15:  end if
16: end while
17: return  $b_i, c_i, g_i \quad \forall j_i \in J$ 

```

V. DISCUSSION

A. Delayed Effectiveness of Cache

When the cache VMs are created for a job, the cache is empty and not filled with the training data at first. During the training of a job, the fetched data from the storage cluster will be saved to the SSD of the cache VMs. However, because DL training will only use a data item once per epoch, the cached data will not be effective (saving bandwidth for the job) until the next epoch. This is called the delayed effectiveness of cache for DL training. Fig. 5 illustrates two cases of the delayed effectiveness. In Fig. 5a, the cache is allocated at the 30% progress in the first epoch. For the rest of the epoch, the newly fetched data can fulfill the newly allocated cache. In the following epochs, all cached data items are effective, leading to a cache hit ratio of 70%. In Fig. 5b, at the 30% progress of the first epoch, a cache of the dataset size is allocated. But the first epoch can only fill it with 70% data. Thus the 2nd epoch only has a cache hit ratio of 70%. The remaining 30% missed data will be fetched in the 2nd epoch. Then starting from the 3rd epoch, the cache hit ratio will be 100%.

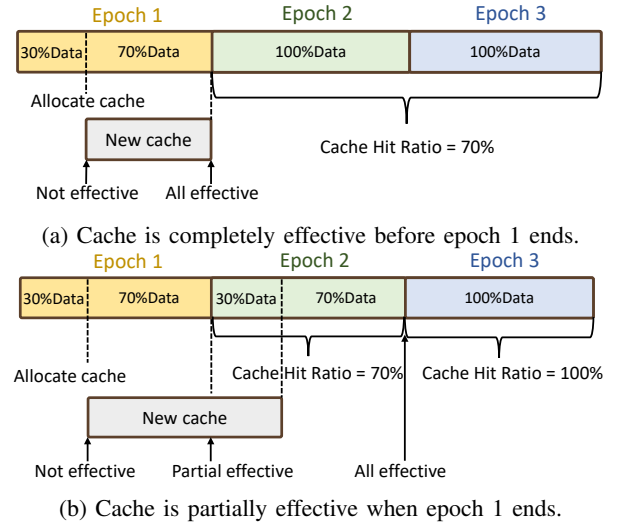


Fig. 5: Two cases for delayed effective cache. In (b), cache is completely effective in the middle of epoch 2. The point where the cache is all effective cannot be obtained in advance due to the random data access pattern of DL.

Previous analysis in Sec. III-B states $T_i = \frac{e_i d_i}{f_i^*(b_i, c_i, g_i)}$. This contains the assumption that the allocated cache is effective immediately. This does not affect the algorithm performance much in the evaluation, since the training of DL will usually take tens or even hundreds of epochs. It has little impact to the overall performance when ignoring the 1 or 2 epochs to make the cached items effective.

B. Online Allocation of CBA and AutoCBA

The optimality proven in Theorem 2 and Theorem 3 is based on the assumption that the allocation cannot be changed after jobs are launched. In practice, the system can adjust the allocation dynamically when a job is running. When the set of jobs changes, CBA or AutoCBA will be executed again to calculate a new allocation to maintain the optimality. Re-computation of the scheduling plan is common for DL schedulers, because the DL training time is usually far longer than the allocation algorithm execution time. The time-complexity of CBA is $O(n)$, whose execution time is negligible. The time-complexity of AutoCBA is $O(I \cdot n)$, where I is the number of iterations. Our evaluation shows it only takes 1 second for AutoCBA to find the allocation for 1000 DL training jobs, which is also negligible for scheduling DL training jobs that usually run for hours and even days.

VI. EXPERIMENTS

We evaluate the proposed algorithms on the production trace from Microsoft Azure's deep learning cluster for one month. A specific analysis of the trace can be found in Sec. VI-A. We first implement our algorithms on a small-scale GPU testbed. To conduct the large-scale experiments, we build a simulator and use the real testbed to validate the high fidelity of the simulator. The key insights are summarized as follows.

- To eliminate the IO bottleneck in **Problem 1**, CBA reduces the total cost by 19.4%, 12.7%, and 20.5% compared with only using cache, only using bandwidth and a proportional allocation policy, respectively.
- In **Problem 2** that requires the joint allocation of GPU, cache and bandwidth, AutoCBA improves the total social welfare by up to $2.27\times$ compared with the state-of-the-art schedulers for DL training (Gandiva [12] and Gavel [13]).

A. Experiment settings

Workloads. We use the one-month production trace from Microsoft's GPU clusters for DL training [20]. The one-month trace consists of 12,150 jobs of popular deep learning models in the domain of Natural Language Processing (NLP) and Computer Vision (CV). The CDF of jobs' execution time under the default allocation policy in Microsoft Azure is shown in Fig. 6a. More than 90% of the jobs take less than a week to complete. Fig. 6b illustrates the job arrival time, which exhibits a very random pattern in the month.

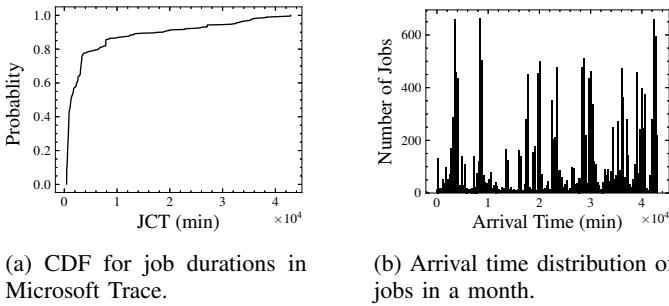


Fig. 6: Trace analysis.

Pricing settings. The prices for unit resources of bandwidth, cache and GPU are from AWS's commonly used instances FSx [7], I3en [3], and P3 [5], respectively. According to the pricing for Amazon FSx, the storage options falls into 7 categories ranging from 50MB/s/TiB to 1000MB/s/TiB. For instance, the service with a minimum bandwidth of 50MB/s/TiB costs 0.000192\$/hour while the service with the highest bandwidth requirements of 1000MB/s/TiB service costs 0.000822\$/hour. The details about the sub-linear bandwidth function β of total required bandwidth and the size of data in the storage service follows the pricing in [7]. The SSD cache cost is 0.000180\$/GB/hour in [3]. The GPU cost in P3 is 3.06\$/hour for one NVIDIA V100 GPU.

Utility Functions. To address the varying degrees of sensitivity to JCT between jobs, each job is assigned with a job-specific utility function when submitted. Similar to previous works [21], [22], we assign the utility function for each job from the following function types and keep it fixed for different baselines. Here t is the JCT of a DL job.

- Linear Function: $U_i(t) = -k_1 t + k_2$
we set $k_1 \in [10, 20]$, $k_2 \in [e_i \frac{d_i}{f_i^*(1)}, 2e_i \frac{d_i}{f_i^*(1)}]$
- All or Nothing (AoN) Function: $U(t) = \begin{cases} k_1 & t < k_2 \\ 0 & t \geq k_2 \end{cases}$
we set $k_1 \in [10, 100]$, $k_2 \in [e_i \frac{d_i}{f_i^*(1)}, 2e_i \frac{d_i}{f_i^*(1)}]$

- Sigmoid Function: $U(t) = \frac{k_2}{e^{k_1(t-k_3)+1} + 1}$
we set $k_1 \in [1, 10]$, $k_2 \in [50, 100]$, $k_3 \in [e_i \frac{d_i}{f_i^*(1)}, 2e_i \frac{d_i}{f_i^*(1)}]$

Jobs with linear utility are latency sensitive and have utilities decrease with longer JCT. While jobs with AoN utility are associated with hard deadlines. If the JCT of a job with AoN utility exceeds its deadline, utility drops to 0. The sigmoid utility is a combination of the above functions. Usually, the jobs using AoN and sigmoid utility functions are used for training production models that require to be completed within a hard or soft duration. Most jobs in the GPU clusters are for research, testing and model exploration, which prefer linear utility function. Therefore, we set the ratio of linear, AoN and sigmoid utility functions to 0.7, 0.1 and 0.2, respectively.

Baselines for CBA. For CBA in **Problem 1**, we have the following three baselines to allocate cache and bandwidth for each job to eliminate IO bottleneck.

- *All Cache*: The cache of the dataset size is allocated for all jobs. Note that for the first epoch of each job, bandwidth is still required to pull the dataset from storage service.
- *All Bandwidth*: Allocate bandwidth to all jobs. The jobs read data directly from the storage service.
- *Prop*: Assign cache and bandwidth proportionally to the number of GPUs allocated for each job by cluster's default policy. We set the cache per GPU to be 45GB and bandwidth per GPU to be 100MB/s according to [5].

Baselines for AutoCBA. Existing cluster schedulers for DL training do not consider the IO bottleneck and thus cannot allocate cache and bandwidth. To have a fair comparison, we enhance them with CBA to eliminate the possible IO bottleneck given their allocation of GPUs. Specifically, each time the baselines allocate new GPUs to jobs, they call CBA to allocate cache and bandwidth resources within the budget. The evaluated baselines for **Problem 2** are listed as follows.

- *Gandiva* [12]: Gandiva prioritizes GPUs to the jobs with the highest throughput growth in a grow-shrink manner.
- *Gavel* [13]: Gavel uses linear programming among jobs to allocate GPU resources, with the goal of maximizing total utility among all jobs within the given budget.
- *Random*: Similar to AutoCBA's iterative allocation but it randomly increases the number of GPUs for a job that can bring positive social welfare within the budget.

B. Fidelity Test on a Real Testbed.

We first build a testbed with two GPU servers, each containing a GeForce RTX 3090 Ti GPU. A separate storage server is used as the storage cluster running Networked File System (NFS) to allow the GPU servers to load data over the network. We implement the bandwidth allocation with Linux's `tc qdisc` [23] to control the network throughput between the compute cluster and the storage cluster speed like how AWS's FSx [7] does. Table IV shows the one-epoch training time of ResNet-50 on ImageNet-1k. On RTX3090 Ti, it requires 320 Mbit/s for the job to load the data without IO bottleneck. When the network is throttled to 40 Mbit/s, the job completion time

(JCT) also takes $8\times$ longer which increases from 4 minutes to over 33 minutes. Our simulator has very high fidelity. The error on the job completion time is up to 9.15% compared to real training. In the rest of the paper, we use this high-fidelity simulator to evaluate our algorithms and the baselines in large-scale clusters.

Bandwidth Limit	Testbed JCT	Simulator JCT
40 Mbit/s	33min48s	31min52s
120 Mbit/s	11min50s	10min45s
160 Mbit/s	7min53s	8min13s
320 Mbit/s	4min3s	4min17s
No limit	4min0.8s	4min17s

TABLE IV: One epoch training time of ResNet-50 on the testbed and simulator.

C. Results and Analysis

Fig. 7 shows that CBA decreases the total cost at the storage level by 19.4%, 12.7% and 20.5% when compared with All Cache, All Bandwidth and Prop. Since CBA distinguishes the different cache and bandwidth requirements of jobs, it allocates the most cost-effective type of resource to each job. In contrast, All Cache baseline cannot benefit from jobs with high bandwidth requirements, and vice versa for All Bandwidth baseline. As for Prop, it allocates a fixed amount of cache and bandwidth to each job, regardless of the job's throughput and dataset size, thus leading to the worst performance.

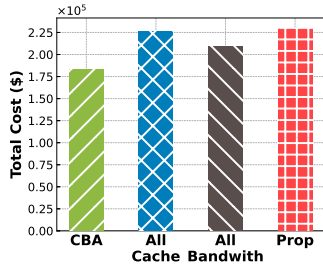


Fig. 7: Storage-level total cost for CBA and other baselines.

To understand why CBA is more efficient than the baselines, we further plot Fig. 8 to illustrate the time variation of the cost on cache and bandwidth. CBA find the best trade-off between cache and bandwidth, and it can find the optimal combination at all time. All Cache in Fig. 8b spent almost all cost on the cache. Note that, due to the delayed effectiveness of cache we explained in Sec. V-A, it still needs to spend some cost on bandwidth to fetch the data from the storage cluster. The cost of bandwidth in All Cache only takes a small portion of the total cost, which further validates our assumption that ignoring the delayed effectiveness in CBA has little impact on the overall cost. All Bandwidth baseline in Fig. 8c directly pays all cost to the cloud service provider to ensure a high enough bandwidth from the storage cluster, which is not efficient when there is a large number of cache-efficient jobs. The Prop baseline in Fig. 8d spent the same cost on cache and bandwidth, which is not flexible. To ensure the

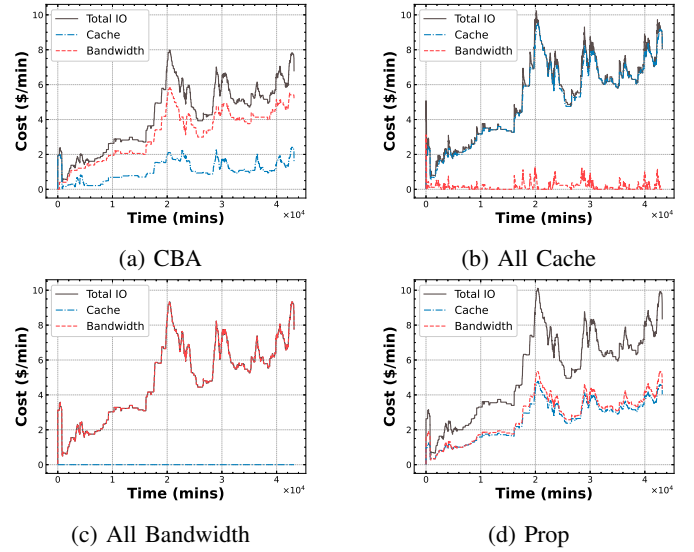


Fig. 8: The cost of CBA and the three baselines for eliminating IO bottleneck in Problem 1.

IO bottleneck elimination for all jobs, some jobs may have over-allocated cache or bandwidth, leading to a waste of cost.

AutoCBA. Fig. 9a shows the social welfare of AutoCBA and the 3 baselines. AutoCBA improves the total social welfare by 1.37x, 2.27x and 2.80x, compared with Gandiva, Gavel and Random, respectively. Fig. 9b and Fig. 9c shows the utility and average JCT of AutoCBA and the 3 baselines. We notice that the improvement of utility and the reduction of average JCT is much lower than the improvement of social welfare. It indicates that AutoCBA has a much efficient cost control to achieve a higher utility with a much lower cost than the baselines, even when the baselines have already been enhanced by CBA. Gandiva adds GPUs to the jobs with highest throughput increase but is unaware of the cost and how long a job will be running. Given a limited budget and online jobs, Gandiva cannot select the most cost-effective jobs. Similarly, Gavel aims at increasing the utility of jobs, without considering the limited budget. It also validates the importance of jointly optimizing GPU, cache and bandwidth allocation in GPU clusters, instead of allocating them separately.

Impact of Budget. In Fig. 9d, we vary the total budget to investigate the performance of AutoCBA and the three baselines. With the increased budget, all algorithms achieve a lower average JCT since the IO bottleneck of more jobs is eliminated via scaling up bandwidth or creating cache VMs. AutoCBA always performs the best under all budget settings compared with the baselines. Moreover, we find AutoCBA has the most improvement over the baselines with the least budget of $\$1.2 \times 10^6$. This is because AutoCBA can plan the budget more carefully to scale GPUs of the jobs that can benefit the most from the cache and bandwidth.

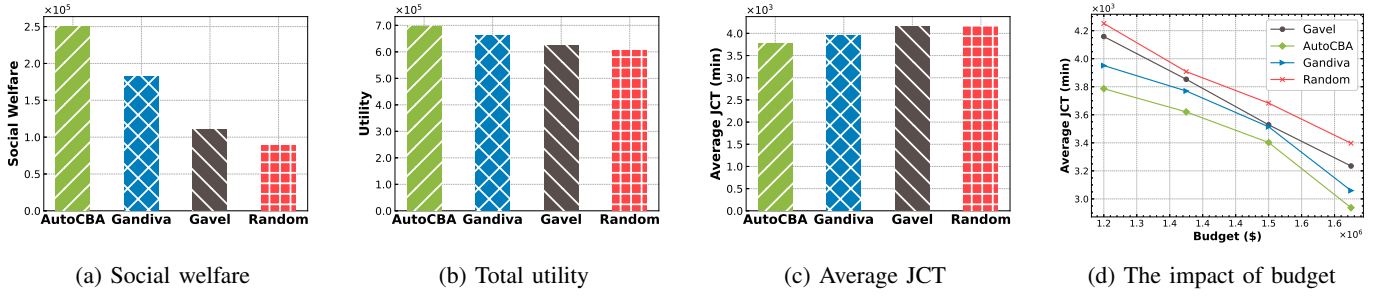


Fig. 9: The performance of social welfare, utility, JCT and sensitivity test of budget.

VII. RELATED WORK

A. Cache Systems for DL training

Existing cache systems for DL training fall into two main categories.

- The first type of system builds its cache into the data loading library. CoorDL [24], for example, statically allocates a certain amount of cache space for each task in a virtual machine or container, and does not allow cache resource sharing among these virtual machines or containers in the whole system.
- The second type of system implements distributed cache, such as Redis, Alluxio [25] and Quiver [14], which put the local cache of all servers into a cache pool that can be shared by all tasks and usually support high-speed access to the disks of peer servers as fast as access to local disks.

These solutions only manage cache without coordinating with the cloud service provider to dynamically adjust the cache space or bandwidth to the remote storage according to the workload preference. CBA and AutoCBA are designed for DL training that can exploit the job preference on cache and bandwidth to allocate them in the most efficient way.

B. Configuration Picking

Configuration picking solutions have been proposed in both data analysis [26], [27] and deep learning [28]. They either build performance models or use profiling to find the best configurations in clouds, e.g., picking the best VM type and the number of VMs. These approaches usually require a large number of trials which is too costly for DL jobs. CBA and AutoCBA consider the resource configuration specifically for DL jobs that jointly chooses GPU, cache and bandwidth. Instead of having a lot of trials, we exploit the special data access pattern of DL training to derive the optimal allocation with a closed-form approach, which is much faster and more cost-efficient compared to existing configuration picking solutions.

C. Schedulers for DL training

The scheduling and resource allocation for DL training has been widely investigated in past works [12], [13], [29]–[31]. Gandiva [12] is the first cluster scheduler for DL training, which aggressively leverages the predictability of DL training to schedule and adjust the GPU allocation during execution. Tiresias [29] extends the idea of the shortest-job-first

policy to allocate GPUs in order to minimize the average JCT. Gavel [13] further extends the scheduling to consider heterogeneous GPU resources and use a unified framework to calculate the scheduling decision by linear programming. These schedulers are efficient because their expected GPU training speed can be very accurate only when IO is not a bottleneck. Moreover, they only focus on allocating GPUs but ignore the impact of cache and bandwidth. CBA improves them by efficiently allocating cache and bandwidth to eliminate the IO bottleneck. AutoCBA further considers the joint optimization of GPU, cache and bandwidth to optimize the cluster efficiency in a unified framework.

VIII. CONCLUSION

In this paper, we study the IO bottleneck problem for DL training jobs in modern clouds with separated compute and storage. We identify the huge cost difference and heterogeneous preference of using more cache and higher bandwidth to alleviate IO bottleneck. To find the most efficient way, we first propose CBA to exploit the heterogeneous job characteristics based on their ideal training throughput and dataset size, to derive the optimal allocation. Moreover, we find the interdependency of dynamic GPU scaling of DL training and the cache-bandwidth allocation. Thus, we further propose AutoCBA to jointly optimize the allocation of GPU, cache and bandwidth to maximize the social welfare for jobs with different utility functions within a limited budget. Real-cluster experiments and large-scale simulation driven by production traces show that CBA can eliminate IO bottleneck of DL training jobs with the least cost, and AutoCBA can greatly improve the social welfare compared with the state-of-the-art schedulers for DL. Our results point out an important direction of taking the storage resource as the first-class citizen, which should be jointly managed with the compute resources (e.g., CPU and GPU). Existing cluster scheduling algorithms optimizing different objectives (e.g., JCT, utilization, fairness) should be re-considered with the limitation of IO bottleneck.

ACKNOWLEDGEMENT

This work is partially supported by the NSFC Grants (62132009, 62072344 and U20A20177) and the Fundamental Research Funds for the Central Universities in China. Haisheng Tan (email: hstan@ustc.edu.cn) and Zhenhua Han (email: hzhua201@gmail.com) are the corresponding authors.

REFERENCES

- [1] "Azure blob storage," <https://azure.microsoft.com/en-us/services/storage/blobs/>.
- [2] "Aws amazon storage," <https://aws.amazon.com/products/storage/>.
- [3] "Amazon ec2 i3en instances," <https://aws.amazon.com/ec2/instance-types/i3en/>.
- [4] "Azure cache for redis," <https://azure.microsoft.com/en-us/services/cache/>.
- [5] "Amazon ec2 p3 instances," <https://aws.amazon.com/ec2/instance-types/p3/>.
- [6] "Azure high-performance computing," <https://azure.microsoft.com/en-us/solutions/high-performance-computing/>.
- [7] "Amazon fsx," <https://aws.amazon.com/fsx/>.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [11] "Bert pre-training," <https://www.deepspeed.ai/tutorials/bert-pretraining/>.
- [12] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, and Q. Zhang, "Gandiva: introspective cluster scheduling for deep learning," in *Operating Systems Design and Implementation*, 2018.
- [13] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, "{Heterogeneity-Aware} cluster scheduling policies for deep learning workloads," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 481–498.
- [14] A. V. Kumar and M. Sivathanu, "Quiver: An informed storage cache for deep learning," in *18th USENIX Conference on File and Storage Technologies (FAST 20)*, 2020, pp. 283–296.
- [15] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [16] S. Abu-El-Haija, N. Kothari, J. Lee, A. P. Natsev, G. Toderici, B. Varadarajan, and S. Vijayanarasimhan, "Youtube-8m: A large-scale video classification benchmark," in *arXiv:1609.08675*, 2016. [Online]. Available: <https://arxiv.org/pdf/1609.08675v1.pdf>
- [17] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
- [18] H. Jégou, F. Perronnin, M. Douze, J. Sánchez, P. Pérez, and C. Schmid, "Aggregating local image descriptors into compact codes," *IEEE transactions on pattern analysis and machine intelligence*, vol. 34, no. 9, pp. 1704–1716, 2011.
- [19] H. Zhao, Z. Han, Z. Yang, Q. Zhang, M. Li, F. Yang, Q. Zhang, B. Li, Y. Yang, L. Qiu, L. Zhang, and L. Zhou, "Silod: A co-design of caching and scheduling for deep learning clusters," in *EuroSys 2023*, May 2023. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/silod-a-co-design-of-caching-and-scheduling-for-deep-learning-clusters/>
- [20] "Analysis of large-scale multi-tenant gpu clusters for dnn training workloads," <https://www.usenix.org/conference/atc19/presentation/jeon>.
- [21] C. Zhang, H. Tan, H. Huang, Z. Han, S. H.-C. Jiang, N. Freris, and X.-Y. Li, "Online dispatching and scheduling of jobs with heterogeneous utilities in edge computing," in *Proceedings of the Twenty-First International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, 2020, pp. 101–110.
- [22] L. Chen, W. Cui, B. Li, and B. Li, "Optimizing coflow completion times with utility max-min fairness," in *IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*. IEEE, 2016, pp. 1–9.
- [23] "tc(8) — linux manual page," <https://man7.org/linux-man-pages/man8/tc.8.html>.
- [24] J. Mohan, A. Phanishayee, A. Raniwala, and V. Chidambaram, "Analyzing and mitigating data stalls in dnn training," 2020.
- [25] H. Li, "Alluxio: A virtual distributed file system," Ph.D. dissertation, UC Berkeley, 2018.
- [26] O. Alipourfard, H. H. Liu, J. Chen, S. Venkataraman, M. Yu, and M. Zhang, "{CherryPick}: Adaptively unearthing the best cloud configurations for big data analytics," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 469–482.
- [27] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica, "Ernest: Efficient performance prediction for {Large-Scale} advanced analytics," in *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, 2016, pp. 363–378.
- [28] Y. Li, Z. Han, Q. Zhang, Z. Li, and H. Tan, "Automating cloud deployment for deep learning inference of real-time online services," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 1668–1677.
- [29] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo, "Tiresias: A {GPU} cluster manager for distributed deep learning," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 485–500.
- [30] A. Qiao, S. K. Choe, S. J. Subramanya, W. Neiswanger, Q. Ho, H. Zhang, G. R. Ganger, and E. P. Xing, "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning," in *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021.
- [31] Z. Han, H. Tan, S. H.-C. Jiang, X. Fu, W. Cao, and F. C. Lau, "Scheduling placement-sensitive bsp jobs with inaccurate execution time estimation," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 1053–1062.